Enterprise Security Architecture

Security Design for a Vulnerability Detection and Remediation Platform

This report explains the security architecture of the enterprise platform. The platform uses Wazuh for vulnerability detection, a backend API for decision and governance, PostgreSQL for traceability, Ansible or Microsoft Graph for controlled remediation, and Wazuh again for verification.

PA Project

May 4, 2026

Contents

1 Project Context **3**

2 Global Secure Architecture **3**

2.1 Component Responsibilities 4

3 Authentication **4**

3.1 Recommended Authentication Mechanisms 4

3.2 Why Authentication Matters 5

4 Authorization and RBAC **5**

4.1 Role-Based Access Control 5

4.2 Asset Scope 5

5 Least Privilege **5**

6 Backend API Security **6**

6.1 Required Controls 6

6.2 Sensitive Endpoint Example 6

7 Input Validation **7**

8 Secrets Management **7**

8.1 Development vs Production 7

9 Database Security **8**

9.1 Database Controls 8

10 Audit Logging **8**

10.1 Why Audit Logs Matter 9

10.2 Audit Log Fields 9

11 Network Segmentation **9**

12 Secure Remediation Model **9**

12.1 Bad Design 10

12.2 Correct Design 10

12.3 Remediation Policy Example 10

13 Approval Workflow **10**

14 Ansible Security **10**

14.1 Security Rules 11

14.2 Example Flows 11

15 Microsoft Graph Security **11**

15.1 Use Cases 11

15.2 Security Rules 11

16 Verification After Correction **12**

17 Docker Security **12**

17.1 Container Layout 12

17.2 Rules 12

18 Minimum Secure Baseline **12**

19 Final Security Statement **13**

20 References **13**

1 Project Context

The project targets an enterprise environment. The laboratory environment is only a proof of concept used to validate the technical flow: Wazuh server, agents, inventory collection, vulnerability detection, API access, and remediation scenarios.

In the enterprise version, the platform must be secure by design because it handles sensitive information and can trigger actions on real company machines.

The platform protects three critical elements:

- **Security data**: vulnerabilities, alerts, assets, logs, and endpoint state.
- **Administrative credentials**: Wazuh API credentials, database secrets, SSH keys, WinRM credentials, Ansible tokens, and Microsoft Graph secrets.
- **Remediation actions**: commands, playbooks, policies, approvals, execution results, and verification status.

Core idea: Wazuh detects, the backend decides, controlled automation corrects, and Wazuh verifies.

2 Global Secure Architecture

The platform is divided into layers. Each layer has a clear responsibility and limited access to the other layers.

```
Enterprise User
    |
    | SSO + MFA
    v
Frontend Web App
    |
    | HTTPS
    v
Backend API
    |
    |-- Authentication and authorization
    |-- Policy engine
    |-- Input validation
    |-- Audit logging
    |
    |-----------------> Wazuh API / Indexer
    |-----------------> PostgreSQL
    |-----------------> Ansible AWX / Automation Platform
    |-----------------> Microsoft Graph / Intune
    |
    v
Remediation Workers
    |
    |-- SSH for Linux
    |-- WinRM for Windows
    |-- Microsoft Graph for Intune-managed Windows
    |
    v
```

```
Enterprise Endpoints
    |
    v
Wazuh rescan and verification
```

2.1 Component Responsibilities

Component	Responsibility	Security Control
Frontend	User interface for dashboards, vulnerabilities, approvals, and remediation tracking	No direct access to Wazuh, database, Ansible, or secrets
Backend API	Decision layer, RBAC, policy checks, data normalization, integration with Wazuh and automation tools	Authentication, authorization, validation, audit logging
PostgreSQL	Stores normalized application data and history	Least-privilege DB user, protected backups, audit tables
Wazuh	Detection and verification engine	Dedicated API user, RBAC, restricted API access
Ansible / AWX	Executes approved remediation tasks	Job templates, controlled variables, credential vaulting
Microsoft Graph	Manages Windows endpoints through Intune and Windows Update for Business	Minimal permissions, app registration, tenant restrictions
Workers	Execute long-running remediation and verification jobs	Isolated execution, limited network access, logged output

3 Authentication

Authentication answers the question:

> *Who are you?*

In an enterprise platform, simple username and password authentication is not enough. The platform should integrate with a trusted identity provider.

3.1 Recommended Authentication Mechanisms

- **SSO**: Single Sign-On using Microsoft Entra ID, LDAP, or Keycloak.
- **MFA**: Multi-Factor Authentication for administrators and approvers.
- **Short-lived tokens**: access tokens expire quickly.
- **Session expiration**: inactive sessions are automatically invalidated.
- **Separate service accounts**: automation accounts are not human accounts.

3.2 Why Authentication Matters

If an attacker steals a password, MFA and token expiration reduce the impact. If a service account is compromised, separation between human users and automation accounts helps contain the incident.

4 Authorization and RBAC

Authorization answers the question:

> *What are you allowed to do?*

Authentication identifies the user. Authorization decides whether that user is allowed to perform a specific action.

4.1 Role-Based Access Control

RBAC means permissions are assigned to roles instead of individual users.

Role	**Allowed Actions**
Viewer	View assets, vulnerability lists, and reports
Analyst	Analyze vulnerabilities and prepare remediation proposals
Approver	Approve sensitive remediation actions
Admin	Manage users, integrations, policies, and platform settings
Automation Worker	Execute technical jobs; not a human role

4.2 Asset Scope

Asset scope limits users to the machines or departments they are responsible for.

Example:

- A user responsible for Finance machines cannot remediate HR machines.
- A user responsible for workstations cannot remediate production servers.

5 Least Privilege

Least privilege means giving every user, service, and integration only the permissions required to perform its job.

Examples:

- The backend should not use the default Wazuh administrator account in production.
- The Wazuh API user should only read agents, inventory, vulnerabilities, and required status data.

- The database user should not be a database superuser.
- The Ansible account should only execute approved remediation actions.

Why it matters: if one account is compromised, least privilege limits the damage.

6 Backend API Security

The backend API is the control center of the platform. Every sensitive request must be verified before execution.

6.1 Required Controls

- HTTPS only.
- JWT validation and expiration.
- RBAC checks on every endpoint.
- Input validation using typed schemas.
- Pagination and result limits.
- Rate limiting on sensitive endpoints.
- Structured errors without stack traces.
- No sensitive data in URLs.

6.2 Sensitive Endpoint Example

Before running a remediation job, the backend must check:

```
POST /remediation-jobs/{id}/run
```

- Is the user authenticated?
- Does the user have the correct role?
- Is the target asset inside the user's scope?
- Is the remediation job approved?
- Is the action allowed by policy?
- Is the target machine reachable?

7 Input Validation

Input validation means verifying all data received by the backend.

For example, if the frontend sends a package name:

```
{
  "package_name": "openssl"
}
```

The backend must verify that the value is expected, safe, and compatible with the selected remediation policy.

This prevents:

- command injection,
- SQL injection,
- unexpected JSON fields,
- invalid package names,
- unauthorized target selection.

8 Secrets Management

Secrets are sensitive values used by the platform.

Secret	**Risk**	**Protection**
Wazuh API password/token	Access to detection data	Dedicated API user, RBAC, rotation
Database password	Access to platform data	Secret manager, limited DB user
SSH private key	Remote access to Linux machines	Vault storage, restricted sudo
WinRM credentials	Remote access to Windows machines	PAM, limited scope
Ansible/AWX token	Ability to launch jobs	AWX RBAC, token rotation
Microsoft Graph secret/certificate	Access to Microsoft tenant operations	Minimal permissions, certificate auth
JWT signing key	Ability to forge application tokens	Secret manager, rotation

8.1 Development vs Production

In development, a local `.env` file is acceptable.

In production, secrets should be stored in a dedicated system such as:

- HashiCorp Vault,
- CyberArk,

- cloud secret manager,
- Docker secrets,
- Ansible Vault for Ansible-specific secrets.

The real `.env` file must never be committed to Git. Only `.env.example` should be committed.

9 Database Security

PostgreSQL stores important application data:

- assets,
- vulnerabilities,
- remediation policies,
- remediation jobs,
- approval requests,
- execution logs,
- audit logs,
- integration configuration.

9.1 Database Controls

- Only the backend can connect to the database.
- The application database user has limited permissions.
- Backups are encrypted.
- Migrations are controlled with Alembic.
- Sensitive fields are encrypted if needed.
- Audit logs are protected against modification.

10 Audit Logging

Audit logging records important actions.

10.1 Why Audit Logs Matter

In an enterprise, we must know:

- who performed an action,
- when the action happened,
- which machine was affected,
- which vulnerability was involved,
- which correction was executed,
- whether the result succeeded or failed.

10.2 Audit Log Fields

```
user_id
user_role
action
target_agent
cve_id
package_name
approval_status
playbook_or_script
started_at
finished_at
exit_code
result
source_ip
```

11 Network Segmentation

Network segmentation means limiting communication between components.

Source	Destination	Port	Purpose
User	Frontend/backend gateway	443	Secure web access
Backend	Wazuh API	55000	Read agents, inventory, vulnerabilities
Backend	PostgreSQL	5432	Application data
Worker	Linux endpoints	22	SSH remediation
Worker	Windows endpoints	5985/5986	WinRM remediation
Agents	Wazuh Manager	1514/1515	Agent data and enrollment

The frontend must not connect directly to the database, Wazuh API, Ansible, or Microsoft Graph.

12 Secure Remediation Model

Auto-correction is the most sensitive part of the project. It must be controlled by policies.

12.1 Bad Design

```
User types a shell command in the browser
Backend runs it on a machine
```

This is dangerous because it creates unrestricted remote command execution.

12.2 Correct Design

```
Vulnerability detected
    -> Backend matches approved remediation policy
    -> Approval is requested if needed
    -> Worker launches approved Ansible playbook or Graph action
    -> Output is logged
    -> Wazuh verifies the result
```

12.3 Remediation Policy Example

```
{
  "name": "Upgrade Debian package",
  "allowed_os": ["debian", "ubuntu"],
  "action_type": "ansible_playbook",
  "playbook": "upgrade_package.yml",
  "requires_approval": true,
  "risk_level": "medium",
  "verification": "wazuh_vulnerability_status"
}
```

13 Approval Workflow

Some corrections can be automatic. Others require approval.

Situation	Decision
Low-risk workstation package update	Auto-correction can be allowed
High severity package on non-critical machine	Approval recommended
Kernel update	Approval and maintenance window required
Firewall change	Approval and rollback plan required
Production server correction	Approval required
Disconnected agent	Correction blocked until reachable

14 Ansible Security

Ansible is the main execution engine for remediation.

14.1 Security Rules

- Use AWX or Ansible Automation Platform job templates.
- Do not allow arbitrary playbook execution.
- Store credentials in AWX or a vault.
- Restrict inventories by environment.
- Allow only controlled variables.
- Log every job output.
- Review playbooks before enabling them.

14.2 Example Flows

```
Linux correction:
Backend -> AWX API -> Ansible playbook -> SSH -> Linux endpoint

Windows correction:
Backend -> AWX API -> Ansible playbook -> WinRM -> Windows endpoint
```

15 Microsoft Graph Security

Microsoft Graph can complement Ansible for Windows machines managed by Intune or Windows Update for Business.

15.1 Use Cases

- list managed Windows devices,
- read compliance status,
- monitor Windows update state,
- interact with Intune policies,
- support enterprise-scale Windows remediation.

15.2 Security Rules

- Use Microsoft Entra ID app registration.
- Grant minimum required permissions.
- Prefer certificate-based authentication in production.
- Restrict actions to specific device groups.
- Separate development and production tenants.
- Log every Graph API action.

16 Verification After Correction

A remediation job is not successful simply because a command finished successfully.

The correct verification flow is:

```
Run remediation
    -> Store execution output
    -> Restart or wait for Wazuh agent inventory scan
    -> Query Wazuh again
    -> Check vulnerability status
    -> Mark job as resolved or failed
```

The source of truth for final validation is Wazuh.

17 Docker Security

Docker helps organize the platform, but containers must also be isolated.

17.1 Container Layout

```
frontend
backend
postgres
redis
worker
```

17.2 Rules

- Frontend talks only to backend.
- Backend talks to PostgreSQL, Redis, Wazuh API, AWX, and Graph.
- PostgreSQL is not exposed publicly.
- Workers have access only to required endpoint networks.
- Secrets are not baked into Docker images.

18 Minimum Secure Baseline

The first enterprise-ready version should include:

- FastAPI backend with authentication.
- RBAC roles.
- PostgreSQL audit logs.
- Dedicated Wazuh API user.

- No direct frontend access to Wazuh or database.
- Policy-based remediation only.
- Ansible job templates.
- Manual approval for risky actions.
- Least-privilege SSH/WinRM accounts.
- Verification through Wazuh after correction.
- Docker network separation.
- `.env.example` committed and real `.env` ignored.

19 Final Security Statement

The platform is designed with security by default: Wazuh provides detection, the backend enforces identity, authorization and policies, Ansible or Microsoft Graph executes only approved remediation actions, and Wazuh verifies the final state after correction.

20 References

- OWASP API Security Project: https://owasp.org/www-project-api-security/
- OWASP Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/
- Wazuh API RBAC: https://documentation.wazuh.com/current/user-manual/api/rbac/index.html
- Wazuh API security: https://documentation.wazuh.com/current/user-manual/api/securing-api.html
- Ansible Automation Controller documentation: https://docs.ansible.com/automation-controller/
- Microsoft Graph documentation: https://learn.microsoft.com/en-us/graph/